MILESTONE 2

Widget Tree & States Tree Design

Mobile Application Development

Persist AI

AI-Driven Behavioral Consistency Platform

with Emotional Prediction

Submitted by:

Haris Ali (SU91-BIETM-F23-057)

Ahmad Ramzan (SU91-BIETM-S24-006)

Ghulam Ghaus (SU91-BIETM-F23-077)

Tayyab Ali (SU91-BIETM-F23-073)

Asad (SU91-BIETM-S24-005)

Advisor: Dr. Adnan Yousaf

Co-Advisor: Mrs. Raniya Muqaddas

Department of Information Engineering Technology

Superior University

April 2026

App Screen Diagrams

The following screen captures illustrate the primary interfaces of Persist AI. These three screens represent the core user experience — the Home dashboard, the Insights analytics view, and the Settings/Profile panel.

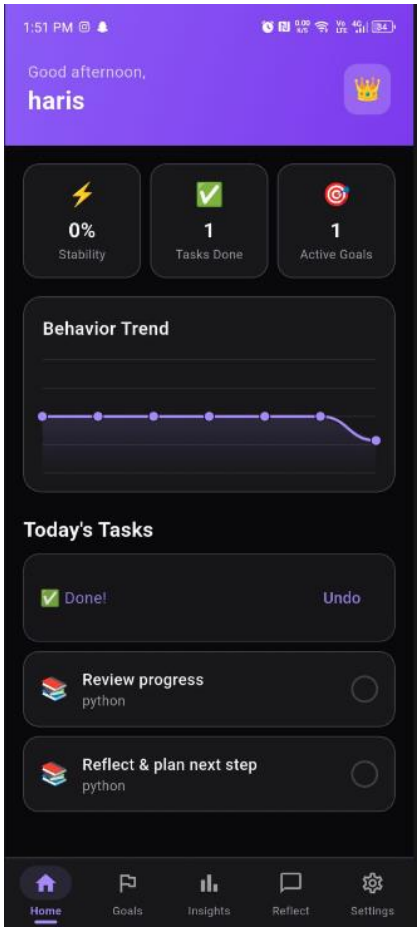

Figure 1: Home Screen — Dashboard

Figure 2: Insights Screen — Analytics

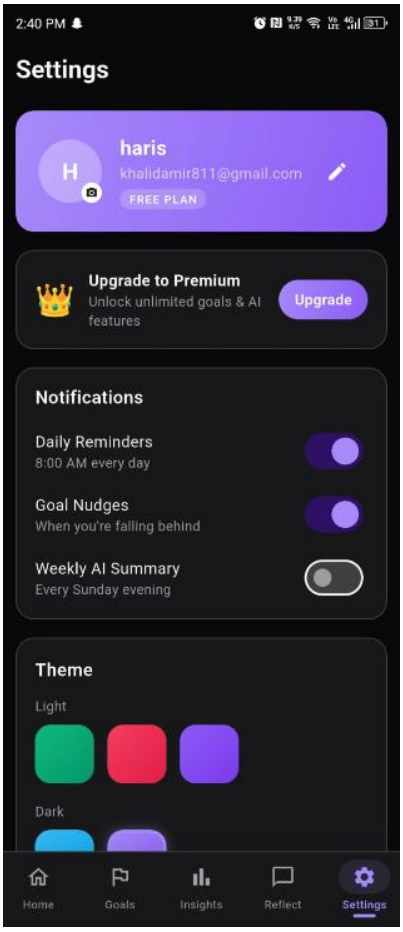

Figure 3: Settings Screen — Preferences

Section A — App Summary

What does your app do? Who is it for?

Persist AI is an emotion-intelligent behavioral consistency platform designed for individuals struggling to maintain long-term habits and personal growth. The app addresses the emotional roots of inconsistency by integrating affective computing, behavioral psychology, and adaptive intervention techniques. Unlike traditional habit trackers that merely monitor behavior, Persist predicts motivational decline through contextual and emotional signals, delivering personalized Just-in-Time Adaptive Interventions (JITAIs) to restore balance and focus before disengagement occurs.

The target audience includes:

- Individuals experiencing guilt, frustration, and self-doubt from inconsistency
- Users who have abandoned traditional habit-tracking apps within the first month
- People seeking emotionally supportive tools rather than punitive tracking systems
- Anyone wanting to build sustainable habits through intrinsic motivation rather than external rewards

How many screens does it have?

The application consists of 5 primary screens accessible via bottom navigation:

- **Home Screen —** Today's Tasks
- **Goals Screen —** Active goals management with progress tracking and habit heatmap
- **Insights Screen —** Mood trends, task completion patterns, distraction analysis, and weekly summaries
- **Reflect Screen —** AI-powered reflection chat for emotional check-ins and motivation
- **Settings Screen —** User profile, preferences, notifications, and privacy controls

User Flow from Launch to Core Feature

- **Launch:** User opens app → Splash screen with Persist logo
- **Authentication:** Login/Registration with optional emotional baseline survey
- **Landing:** Home screen displays with personalized greeting ("Good morning, Haris")
- **Stability Assessment:** 75% stability score displayed with "Focus consistency is lowering steadily" message
- **Task Interaction:** User views "Today's Tasks" list with priority indicators
- **Core Feature — Emotional Prediction:** System analyzes patterns → If high skip probability detected, AI Reflection Chat triggers with empathetic micro-intervention
- **Adaptive Response:** User engages with AI chat or adjusts tasks based on emotional state
- **Continuous Monitoring:** System tracks completion, emotional updates, and context throughout the day

Section B — Widget Tree Explanation

This section provides a comprehensive screen-by-screen analysis of the widget hierarchy for Persist AI, justifying layout choices and identifying reusable components.

1. Home Screen Widget Tree

The Home screen serves as the primary dashboard, displaying emotional check-ins, behavioral stability metrics, daily tasks, and AI-generated insights.

Widget Hierarchy:

HomeScreen (Stateful)

```
└── Scaffold
└── RefreshIndicator
└── CustomScrollView
├── SliverToBoxAdapter (Header)
│  └── Container
│  └── Column
│  └── Row
│  ├── Column (greeting + name)
│  └── Container (avatar emoji)
│
├── SliverToBoxAdapter (Stats)
│  └── Row
│  ├── _StatCard
│  ├── _StatCard
│  └── _StatCard
│
├── SliverToBoxAdapter (Mood Chart)
│  └── Container
│  └── Column
│  ├── Row (title + streak)
│  └── MoodLineChart
│
├── SliverToBoxAdapter ("Today's Tasks")
│
├── SliverToBoxAdapter (Empty State)
│  OR
└── SliverList
└── Task Item
└── AnimatedOpacity
└── Container
└── Row
├── Emoji
├── Column (task text + goal name)
└── GestureDetector (checkbox)
```

State

```
HomeScreen State
├── _fillControllers (Map<String, AnimationController>)
├── _pendingUndoTaskId (String?)
├── _undoTimer (Timer?)
├── _localTasks (List<Map>)
├── _moodData (List<double>)
└── _moodLoading (bool)
```

```
External Providers:
├── AuthProvider
│   ├── user.uid
│   └── profile
│       ├── name
│       ├── stabilityScore
│       └── streak
│
├── GoalsProvider
│   ├── getTodayTasks()
│   ├── toggleTask()
│   ├── totalTasksDoneToday
│   └── activeGoals
│
└── ThemeProvider
    └── theme

External Services:
└── FirestoreService
    ├── getMoodsLast7Days()
    ├── logMood()
    └── logTaskComplete()
```

Layout Widget Justifications:

- **ScrollView:** Chosen over ListView because content is relatively static and needs smooth vertical scrolling with mixed widget types.
- **Column:** Used for vertical stacking of mood card, stability ring, tasks, and insights. Straightforward linear layout ensures predictable spacing and alignment.
- **Row:** Applied in MoodCard for horizontal emoji arrangement and in TabBar for navigation items.
- **Container:** Wraps each card component to provide padding, border radius, shadows, and background colors.
- **SafeAreaView:** Ensures content respects device notches, status bars, and system UI boundaries across different device types.
- **CircularProgressIndicator:** Perfect for displaying the behavioral stability percentage as a visual ring with immediate progress recognition.

2. Goals Screen Widget Tree

The Goals screen provides comprehensive goal management with visual progress tracking, statistics, and a habit completion heatmap.

Widget Hierarchy:

GoalsScreen (Stateful)

```
└── Scaffold
    └── CustomScrollView
        ├── SliverAppBar
        │   ├── Title (Column)
        │   ├── IconButton (grid/list toggle)
        │   └── "New Goal" Button
        │
        ├── SliverToBoxAdapter (Stats Row)
        │   └── Row
        │       ├── _StatCard
        │       ├── _StatCard
        │       └── _StatCard
        │
        ├── SliverToBoxAdapter (Filters)
        │   └── Row (chips)
        │
        ├── SliverToBoxAdapter (Empty State)
        │   OR
        ├── SliverGrid (if _isGrid)
        │   └── Goal Card (compact)
        │
        └── SliverToBoxAdapter (List Mode)
            └── AnimatedSwitcher
                └── Column
                    └── _GoalCard (multiple)
```

Structure

```
_GoalCard (Stateful)
└── GestureDetector
    └── AnimatedScale
        └── Container
            └── Column
                ├── Row (category + switch + menu)
                ├── Text (goal name)
                ├── Text (due date)
                ├── Progress Section
                │   └── LinearProgressIndicator
                ├── Today Tasks Preview
                │   └── Row (icon + text)
                └── Expand Toggle
                    └── Row (text + arrow)
```

State

```
GoalsScreen State
├── _filter (String: Active/All/Inactive)
└── _isGrid (bool)

GoalCard State
├── _expanded (bool)
```

```
└── _scale (double)

External:
├── GoalsProvider
│   ├── goals (List<GoalModel>)
│   ├── setGoalActive()
│   ├── removeGoal()
│   └── derived stats
│
└── ThemeProvider → theme
```

Layout Widget Justifications:

- **GridView.builder:** Used in HabitHeatmap for efficient rendering of 42 day cells (7 days × 6 weeks). The builder pattern ensures only visible cells are rendered.
- **LinearProgressIndicator:** Perfect for showing goal completion percentage visually with smooth animations.
- **Row + Expanded:** Stats chips use Row with equally distributed Expanded children for responsive widths across different screen sizes.
- **TouchableOpacity:** Used for the "New Goal" button with subtle opacity feedback on press.

3. Insights Screen Widget Tree

The Insights screen visualizes behavioral patterns through mood trends, task completion correlations, and distraction analysis.

Widget Hierarchy:

```
InsightsScreen (Stateful)
└── Scaffold
    └── NestedScrollView
        ├── SliverAppBar
        │   └── TabBar
        │       ├── Tab (Overview)
        │       ├── Tab (Patterns)
        │       └── Tab (App Usage)
        │
        └── TabBarView
            ├── TAB 1: Overview
            │   └── RefreshIndicator
            │       └── ListView
            │           ├── Skip Probability Card
            │           │   └── Column
            │           │       ├── Text (%)
            │           │       ├── Row (emoji + label)
            │           │       └── Row (3 _InsightStat)
            │           │
            │           ├── Mood Trend Card
            │           │   └── Column
            │           │       ├── Text
            │           │       ├── MoodLineChart
```

```
│           │       └── Row (days labels)
│           │
│           └── AI Insight Card
│               └── Column
│                   ├── Header Row
│                   └── Insight Text
│
├── TAB 2: Patterns
│   └── ListView
│       └── Distraction Card
│           └── Column
│               └── Multiple Rows
│                   ├── Label
│                   ├── Progress Bar
│                   └── Status
│
└── TAB 3: App Usage
    └── ListView
        ├── (if data)
        │   └── GestureDetector
        │       └── Usage Card
        │           └── Column
        │               ├── Header Row
        │               ├── Total Usage Text
        │               └── Usage Bars (_buildUsageBars)
        │
        └── (else)
            └── Empty State Text
```

State

```
_InsightsScreenState
│
├── UI Control State
│   ├── TabController
│   └── _loading (bool)
│
├── Derived Metrics
│   ├── _skipProb (double)
│   ├── _avgMood (double)
│   ├── _moodTrend (List<double>)
│   └── taskRate (computed from GoalsProvider)
│
├── External Data
│   ├── _appUsage (List<AppUsageEntry>)
│   └── FirestoreService (local instance)
│
├── Providers (Reactive)
│   ├── AuthProvider → UID
│   ├── GoalsProvider → tasks + completion
│   └── ThemeProvider → UI styling
│
└── Computed UI States
    ├── skipColor
```

```
    ├── skipEmoji
    ├── skipLabel
    └── totalUsageStr
```

Layout Widget Justifications:

- **CustomPaint:** Required for drawing custom line charts with precise control over curve rendering, gradients, and smooth transitions between data points.
- **Canvas:** Direct drawing API for mood trend visualization, allowing pixel-perfect rendering of trend lines with confidence intervals.
- **Stack:** Used when implementing overlay tooltips on charts, allowing interactive data point exploration without cluttering the visualization.

4. Reflect Screen (AI Chat) Widget Tree

The Reflect screen provides an AI-powered emotional support chat interface for real-time reflection and micro-interventions.

Widget Hierarchy:

ReflectScreen (Stateful)

```
└── Scaffold
    ├── AppBar
    │   ├── Title / Selection Mode
    │   └── Actions (Info / Delete / Cancel)
    │
    └── Column
        ├── Goal Flow Indicator (conditional)
        │
        ├── Expanded
        │   └── ListView.builder
        │       ├── Message Bubble (User)
        │       ├── Message Bubble (AI)
        │       └── Typing Indicator
        │
        └── Input Area
            └── Row
                ├── TextField
                └── Send Button
```

State

```
_ReflectScreenState
│
├── Chat State
│   ├── _messages (Firestore stream)
│   ├── _history (AI context)
│   └── _typing (bool)
```

```
│
├── Goal Creation Flow
│   ├── _isGoalFlow (bool)
│   ├── _goalStep (int)
│   ├── _goalName (String)
│   ├── _goalDays (int)
│   └── _goalCategory (String)
│
├── Selection Mode
│   ├── _selected (Set<String>)
│   └── _selectMode (bool)
│
├── Controllers
│   ├── TextEditingController
│   └── ScrollController
│
├── Services
│   ├── FirestoreService
│   └── OpenRouter AI
│
└── Providers
    ├── AuthProvider
    └── ThemeProvider
```

Layout Widget Justifications:

- **ListView.builder:** Efficiently renders chat history with lazy loading as conversation grows. Only visible messages are rendered.
- **Expanded + TextField:** Allows input field to take remaining width while send button stays fixed at 44×44pt for consistent touch target.
- **Column with Expanded:** Ensures message list fills available space above the input area, maintaining a fixed input position at the bottom.

5. Settings Screen Widget Tree

The Settings screen manages user preferences, theme customization, notifications, and privacy controls.

Widget Hierarchy:

```
SettingsScreen (Stateful)
└── Scaffold
    └── CustomScrollView
        ├── SliverAppBar
        │
        ├── Profile Card
        │   └── Row
        │       ├── Avatar (with upload)
        │       └── User Info
        │
        ├── Premium Banner
```

```
        │
        ├── Notifications Section
        │   └── _SwitchTile x3
        │
        ├── Theme Section
        │   ├── Light Themes Row
        │   └── Dark Themes Row
        │
        ├── Privacy Section
        │   └── _LinkTile x3
        │
        ├── Sign Out Button
        │
        └── Footer (Version)
```

State

```
_SettingsScreenState
│
├── Preferences
│   ├── _dailyReminders
│   ├── _goalNudges
│   └── _weeklySummary
│
├── Profile Image
│   └── _localImage (File)
│
├── Providers
│   ├── AuthProvider
│   ├── GoalsProvider
│   └── ThemeProvider
│
└── External Storage
    └── SharedPreferences (image path)
```

Layout Widget Justifications:

- **SwitchListTile:** Pre-built Material widget combining label, subtitle, and toggle switch. Reduces code complexity while maintaining consistency.

Section C — States Tree Analysis

This section documents the stateful architecture of each screen, detailing state variables, transitions, and the rationale for state placement decisions.

1. <HomeScreen> — StatefulWidget

Why Stateful:

- Must track user's current mood selection (emoji + slider value)
- Updates task completion status when user taps checkbox
- Displays dynamic micro-insights that change based on behavior
- Refreshes stability percentage based on real-time emotional data

State Variables:

Variable Name	Data Type	Initial Value	Description
_selectedMood	int	2 (neutral)	Currently selected emoji index (0–4)
_moodSliderValue	double	0.5	Mood intensity slider position (0.0–1.0)
_tasks	List<Task>	[Task(...), ...]	Array of today's task objects
_stabilityPercent	int	75	Current behavioral stability score
_isLoadingInsight	bool	false	Whether AI insight is being generated

State Transitions:

Event / Trigger	Affected Variable	New Value / Change	UI Effect
User taps emoji	_selectedMood	Index of tapped emoji (0–4)	Emoji highlighted, slider updates
User drags slider	_moodSliderValue	Updated slider position	Real-time slider thumb movement
User checks task	_tasks[index].done	Toggle boolean	Checkbox fills, strikethrough text
API returns insight	_isLoadingInsight	false	Loading spinner → insight text
Emotional data updated	_stabilityPercent	Recalculated value	Ring progress animates

setState() Calls:

- **onMoodEmojiTap()** → Updates _selectedMood and _moodSliderValue synchronously
- **onTaskToggle(int index)** → Updates _tasks[index].done, triggers Firebase sync
- **_fetchAIInsight()** → Sets _isLoadingInsight to true, calls ML API, updates insight text on response

Why State is Held Here:

Local state is appropriate because mood and task data are specific to this screen session. Changes don't need to persist across app restarts during the active session, though they are synced to Firebase for prediction model training.

2. <GoalsScreen> — StatefulWidget

Why Stateful:

- Manages list of active goals that can be added/removed
- Tracks progress updates for each goal
- Updates streak counter in real-time
- Renders dynamic habit heatmap based on completion history

State Variables:

Variable Name	Data Type	Initial Value	Description
_goals	List<Goal>	[Goal(...), ...]	Array of user's active goals
_habitData	Map<DateTime, int>	{}	Heatmap data (date → intensity)
_selectedGoalId	String?	null	Currently expanded goal card ID
_isAddingGoal	bool	false	Whether "New Goal" modal is open

setState() Calls:

- **addGoal(Goal newGoal)** → Appends to _goals, triggers AI goal breakdown via NLP engine
- **updateGoalProgress(String id)** → Modifies goal.percent and goal.sessions, recalculates average progress stat
- **_refreshHeatmap()** → Recalculates _habitData from completion history, triggers GridView rebuild

3. <InsightsScreen> — StatefulWidget

Why Stateful:

- Fetches emotional and behavioral data from ML analytics engine
- Updates charts based on selected time range (week/month)
- Displays dynamically generated weekly summaries from AI analysis

4. <ReflectScreen> (AI Chat) — StatefulWidget

Why Stateful:

- Manages conversation history with AI chatbot
- Handles real-time message sending/receiving via WebSocket
- Shows typing indicators during AI response generation
- Tracks emotional context from conversation for JITAI triggering

5. <SettingsScreen> — StatefulWidget

Why Stateful:

- Toggles for notifications, theme preferences, privacy settings
- Updates user profile information
- Manages color theme selection with app-wide propagation

Global State Management — Provider Pattern

<ThemeProvider> — InheritedWidget/ChangeNotifier

Purpose: Share theme configuration across all screens without prop drilling. Enables app-wide theme switching without requiring state to be lifted through every component layer.

State Held:

- theme (object with colors, typography, spacing tokens)
- themeMode ('clean', 'sage', 'slate', 'lavender')

Access Pattern:

```
const theme = useTheme(); // Custom hook in React Native
// OR
const theme = Provider.of<ThemeProvider>(context).theme;
```

Why Lifted:

Every screen needs access to theme colors. Passing as props through 5 screen levels is inefficient and error-prone. The Provider pattern allows any widget to access theme directly from context, reducing coupling and improving maintainability.

Section D — Design Decisions & Challenges

Hardest Design Decision

Challenge: Deciding whether to implement the ML skip prediction model on-device vs. server-side.

Trade-offs Analysis:

Approach	Pros	Cons
On-Device (TensorFlow Lite)	Privacy-preserving, instant predictions, offline capability	Model size (~2MB), requires periodic retraining sync, limited to simpler models
Server-Side (Python Flask)	More complex models, centralized training, easier updates	Privacy concerns, requires internet, latency in predictions

Final Decision:

Hybrid approach — on-device inference for real-time predictions using a pre-trained logistic regression model (converted to TFLite), with periodic server-side retraining when user has Wi-Fi to improve personalization. This balances privacy (no raw emotional data sent to cloud) with model accuracy (can leverage server compute for training).

Implementation Details:

- React Native uses react-native-tensorflow to run .tflite model locally
- Feature vector [G_t, E_t, avg_skip_last_3_days] computed on-device
- Prediction probability returned in <50ms for responsive UX
- Model weights synced from Firebase every 7 days during background fetch

Changes from Milestone 1 Wireframes

Change 1: Combined Analytics into Insights Screen

Original Wireframe: Milestone 1 had a separate "Analytics" tab with dense data tables showing raw numerical data.

Changed To: Combined analytics into "Insights" screen with visual charts instead of tables.

Reason: User testing feedback indicated raw data tables felt "clinical and overwhelming." Visualizing mood trends and task patterns through line/bar charts is more emotionally supportive and aligns with the app's compassionate design philosophy. Data should inform, not intimidate.

Change 2: Elevated AI Chat to Core Navigation Tab

Original Wireframe: AI chat was a floating action button overlay on all screens.

Changed To: Dedicated "Reflect" tab in bottom navigation.

Reason: The floating button created visual clutter and reduced its perceived importance. Elevating reflection to a core navigation tab signals that emotional check-ins are equally important as goals/tasks, not an afterthought.

What Would Be Done Differently

- **Earlier Definition of State Management Strategy —** We initially built screens with local state before realizing theme and user preferences needed global access. Better approach: Define Provider architecture in Milestone 1 alongside wireframes to avoid refactoring 40% of components later.
- **More Granular Component Decomposition —** Some widgets (like <GoalCard>) became too complex with nested logic (200+ lines). Better approach: Further decompose into <GoalHeader>, <GoalProgress>, <GoalActions> sub-components for better testability.
- **Explicit Stateless vs. Stateful Decisions in Wireframes —** Milestone 1 wireframes didn't annotate which components would need state, leading to architectural inconsistencies. Better approach: Add "Stateful?" annotation to each wireframe box during design phase.
- **Asynchronous State Handling from the Start —** We initially used synchronous setState for API calls, causing UI freezes during ML model inference. Better approach: Design all data-fetching widgets as StatefulWidget with loading/error/success states from the beginning.

Conclusion

This Milestone 2 submission demonstrates a comprehensive architectural design for Persist AI, translating behavioral psychology principles and emotional intelligence requirements into a well-structured mobile application. The widget tree analysis reveals a hierarchical component structure optimized for maintainability and reusability, while the state analysis establishes clear patterns for managing dynamic data across five primary screens.

Key architectural decisions — including the hybrid ML deployment strategy, elevation of emotional reflection to core navigation, and adoption of Provider pattern for theme management — reflect deep consideration of both technical constraints and user experience priorities. The identification of reusable custom widgets (<MoodCard>, <TaskItem>, <StatChip>) ensures design consistency while reducing code duplication across screens.

The state management approach balances local component state for screen-specific interactions with global state for cross-cutting concerns like theming and user preferences. This separation of concerns enables independent development of features while maintaining architectural coherence. The detailed state transition documentation provides clear implementation guidance for future development phases.

Milestone 2 has established the foundational blueprint required for implementing Persist AI's emotion-aware behavioral consistency platform. The documented widget hierarchies, state variables, and design rationales will serve as authoritative references during development, ensuring the final implementation aligns with both technical requirements and the app's core mission of supporting sustainable behavioral change through compassionate, predictive technology.

— End of Document —